

Universidad Pontificia Comillas

ETSI-ICAI

Laboratorio Robots autónomos

PRIMEROS PASOS CON ROS 2

Lydia Ruiz Martínez, Pablo Tuñón Laguna, Jorge Vančo Sampedro



December 2024

¿Cuál es la versión y el nombre de la distribución de ROS instalada en el contenedor que se le ha proporcionado? ¿Cuándo finaliza su soporte? ¿Por qué?

La versión de ROS instalada es la 2a, las distribuciones usadas son Humble y Jazzy. El soporte de Humble finaliza en mayo de 2027 y el de Jazzy en mayo de 2029 (en el día internacional de las tortugas). Ambas distribuciones surgieron en años pares (2022 y 2024 respectivamente) por lo que poseen el soporte extendido de 5 años.

```
printenv ROS_VERSION # Extrae la versión de ROS
printenv ROS_DISTRO  # Extrae la distribución de ROS
```

¿Qué otro comando se puede ejecutar con `ros2 node`?

El primer comando inicializa un nodo de ROS2 que contiene información de una tortuga. El segundo comando muestra los nodos que se encuentran actualmente en ejecución y el tercer comando muestra información sobre el nodo de la tortuga.

```
ros2 run turtlesim turtlesim_node # Inicializa un nodo de ROS2
ros2 node list # Muestra los nodos en ejecución
ros2 node info /turtlesim # Muestra información sobre el nodo de la
tortuga
```

¿Cómo se llama el tema (topic) donde `/teleop_turtle` publica los comandos de movimiento?

El tema se llama `/turtle1/cmd_vel`, obtenido a través de la herramienta `rqt_graph`.

```
rqt_graph # Muestra un gráfico de nodos y temas en ROS2
```

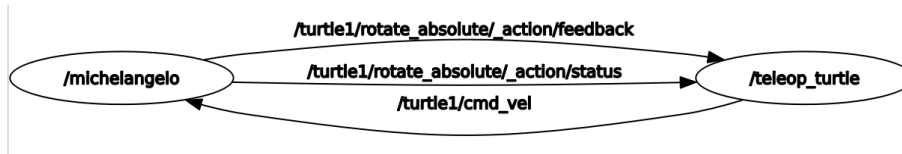


Figura 1: Gráfico de nodos y temas en ROS2 usando rqt_graph, autoría propia

¿Qué recibe /michelangelo de /teleop_turtle?

No recibe nada ya que no todavía no se ha publicado nada, cuando se va a la terminal del nodo teleoperador y se pulsa una tecla, se muestra el mensaje que se envía a través del topic. El mensaje es de tipo `geometry_msgs/msg/Twist` y contiene la velocidad lineal y angular de la tortuga.

```

ros2 topic echo /turtle1/cmd_vel # Muestra los mensajes publicados en
    un topic
ros2 topic info /turtle1/cmd_vel # Muestra información sobre un topic
  
```

¿Qué tipos de datos se intercambian entre el nodo de teleoperación y TurtleSim?

Se intercambian 2 topics con distintos mensajes y una acción:

- `/turtle1/cmd_vel`: Mensaje de tipo `geometry_msgs/msg/Twist` que contiene la velocidad lineal y angular de la tortuga como vectores de 3 elementos float64 (x, y, z).
- `/turtle1/rotate_absolute/_action/status`: Mensaje de tipo `action_msgs/msg/GoalStatusArray` que contiene el estado de la acción como un array de enteros.
- `/turtle1/rotate_absolute/_action/feedback`: acción de tipo Feedback.

```

ros2 topic info /turtle1/cmd_vel # Muestra información sobre un topic
ros2 interface show geometry_msgs/msg/Twist # Muestra información sobre
    un mensaje
ros2 topic info /turtle1/rotate_absolute/_action/status # Muestra
    información sobre un topic
  
```

```
ros2 interface show action_msgs/msg/GoalStatusArray # Muestra  
información sobre un mensaje
```

¿Qué campos hay que completar en package.xml? ¿Qué es exactamente una licencia en este contexto? ¿Puede citar algunas licencias de software?

Hay que completar:

- description: Descripción del paquete.
- maintainer: Nombre y correo electrónico del mantenedor del paquete.
- license: Licencia del paquete.
- exe_depends: Dependencias del paquete, rclpy y std_msgs.

En este contexto, una licencia es un contrato que establece los términos y condiciones bajo los cuales se puede usar, modificar y distribuir un software. En concreto la Apache 2.0 es una licencia de software libre y de código abierto que permite a los usuarios utilizar, modificar y distribuir el software siempre y cuando se de crédito al autor original y se incluya la licencia en cualquier copia o modificación del software. Otras licencias son la GPL, MIT, BSD.

¿Qué hay que añadir a setup.py?

Hay que añadir:

- maintainer: Nombre y correo electrónico del mantenedor del paquete.
- maintainer_email: Correo electrónico del mantenedor del paquete.
- description: Descripción del paquete.
- license: Licencia del paquete.
- entry_points: Punto de entrada del paquete (desde donde se ejecuta) tanto para el publisher como para el subscriber.

¿Cuáles son los pasos para construir y ejecutar un nodo?

Los pasos son:

- Crear un workspace de ROS2.
- Crear un paquete.
- Diseñar los nodos y mensajes necesarios.
- Modificar el package.xml para añadir el mantenedor, la descripción, la licencia y las dependencias.
- Modificar el setup.py para añadir el mantenedor, la descripción, la licencia y el punto de entrada.
- Compilar el paquete.
- Ejecutar el nodo.

```
mkdir -p ~/ros2_ws/src # Crea un workspace de ROS2
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python <package_name> # Crea un
    paquete
# Programar los nodos y mensajes necesarios
# Modificar package.xml y setup.py
colcon build # Compila el paquete
source ~/ros2_ws/install/setup.bash # Añade el paquete al workspace
ros2 run <package_name> <node_name> # Ejecuta el nodo
```

En sus propias palabras, explique brevemente cómo se comunica el nodo parlante (talker) con el oyente (listener)

El nodo publisher, publica un mensaje en un tópico (un canal de comunicación), el nodo subscriber está suscrito a dicho topic de tal manera que cuando se publica un mensaje en dicho canal, el nodo subscriber recibe el mensaje y lo procesa.

Utilice las herramientas de terminal de ROS 2 y rqt para descubrir los nodos que operan el TurtleBot3, así como los publicadores y suscriptores que tienen. Explore también la estructura y el contenido de los mensajes

e intente explicar con sus palabras para qué sirve cada nodo a partir de esa información. Ignore todos los temas (topics) que empiecen con el prefijo rcl_

Existen 4 nodos en ejecución: `/turtlebot3_node`, `/diff_drive_controller`, `/robot_state_publisher` y `/ld08`.

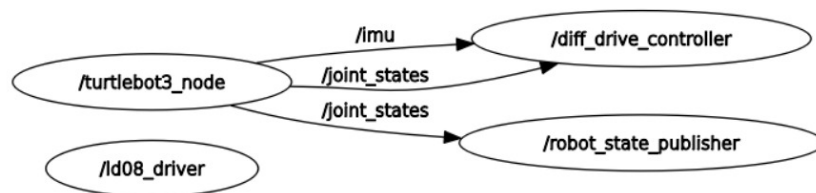


Figura 2: Gráfico de nodos y temas en ROS2 usando `rqt_graph` a través de una terminal con protocolo X11, autoría propia.

Descripción de los nodos:

- `/turtlebot3_node`: Controla el movimiento del TurtleBot3 y publica el topic `/imu` y `/joint_states`.
- `/diff_drive_controller`: Controla el movimiento de las ruedas del TurtleBot3 y se suscribe a los topics `/imu` y `/joint_states`.
- `/robot_state_publisher`: Publica el estado del robot y se suscribe al topic `/joint_states`.
- `/ld08`: Publica la distancia de los objetos detectados por el sensor LD08, este nodo de momento no publica ni se suscribe a ningún topic.

Posteriormente se obtuvieron los topics a través de la terminal con el comando `ros2 topic list`:

- `/battery_state`: Topic con mensaje de tipo `sensor_msgs/BatteryState` que contiene información sobre el estado de la batería.
- `/cmd_vel`: Topic con mensaje de tipo `geometry_msgs/Twist` que contiene la velocidad lineal y angular del robot.

- `/imu`: Topic con mensaje de tipo `sensor_msgs/Imu` que contiene la orientación y velocidad angular del robot.
- `/joint_states`: Topic con mensaje de tipo `sensor_msgs/JointState` que contiene la posición de las ruedas del robot.
- `/magnetic_field`: Topic con mensaje de tipo `sensor_msgs/MagneticField` que contiene la intensidad del campo magnético.
- `/odom`: Topic con mensaje de tipo `nav_msgs/Odometry` que contiene la posición y orientación del robot.
- `/parameter_events`: Topic con mensaje de tipo `rcl_interfaces/msg/ParameterEvent` que contiene los parámetros modificados por el evento.
- `/robot_description`: Topic con mensaje de tipo `std_msgs/msg/String` que contiene la descripción del robot.
- `/rosout`: Topic con mensaje de tipo `rcl_interfaces/msg/Log` que contiene información sobre los logs del sistema.
- `/scan`: Topic con mensaje de tipo `sensor_msgs/LaserScan` que contiene la información del lidar LD08.
- `/sensor_state`: Topic con mensaje de tipo `sensor_msgs/Range` que contiene la distancia de los objetos detectados por el lidar.
- `/tf`: Topic con mensaje de tipo `tf2_msgs/TFMessage` que contiene la información de la transformación entre los distintos frames del robot.
- `/tf_static`: Topic con mensaje de tipo `tf2_msgs/TFMessage` que contiene la información de la transformación estática entre los distintos frames del robot.

¿Qué criterio de signos utiliza el TurtleBot3 para la velocidad angular ω por defecto? ¿Coincide con el habitual?

Después de ejecutar el nodo de teleoperación y desplazar el robot, se observa la siguiente información en la terminal:

```
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity 0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity -0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity 0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity -0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity -0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity 0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity 0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity 0.1
currently: linear velocity 0.0 angular velocity 0.0
currently: linear velocity 0.0 angular velocity -0.1
currently: linear velocity 0.0 angular velocity 0.0
```

Figura 3: Resultado tras el movimiento del TurtleBot3 ejecutado el nodo de teleoperación, autoría propia.

Dado que el robot se desplaza en sentido horario, la velocidad angular ω es positiva. El criterio de signos utilizado por el TurtleBot3 para la velocidad angular es el siguiente: si ω es positiva, el robot se desplaza en sentido horario y si ω es negativa, el robot se desplaza en sentido antihorario. Si el robot siguiera el criterio habitual (el de la mano derecha), la velocidad angular sería negativa para el sentido horario y positiva para el sentido antihorario. Por todo ello se concluye que el criterio del robot no coincide con el habitual.